

Dane, napisy, pliki

Python na maturę z informatyki · skrypt do pracy samodzielnej · 135 minut

Plan (orientacyjnie):

Część	Czas
Przygotowanie	10 min
Temat 1. Wejście, wyjście, formatowanie	20 min
Temat 2. Napisy	25 min
Temat 3. Pliki	30 min
Temat 4. Listy i krotki	25 min
Temat 5. Pętle i wygodne konstrukcje	20 min
Wysłanie plików	5 min

Zadanie dodatkowe na końcu skryptu robisz i wysyłasz w domu (albo zaczynasz, jeśli zostanie czas).

Przygotowanie

1. Utwórz folder, np. `zajecia1`, i skopiuj do niego **4 pliki z danymi**: `liczby.txt`, `uczniowie.txt`, `dane liniowe.txt`, `tekst.txt`.
2. Wszystkie swoje programy zapisuj **w tym samym folderze** co pliki z danymi.
3. Każde ćwiczenie to **osobny plik** o nazwie według wzoru `tT_cwN.py`, np. `t3_cw2.py` to temat 3, ćwiczenie 2. (Jeśli ćwiczenia nie są rozbudowane, możesz zrobić wszystkie ćwiczenia z danego tematu w jednym pliku `tT.py`. W takim przypadku koniecznie zaznacz w kodzie komentarzem lub na terminalu poprzez `print()` którego ćwiczenia dotyczy kod, np.:

```
# Ćwiczenie 1
a = 2
print(a)

# Ćwiczenie 2
...
```

! Na maturze musicie pilnować nazewnictwa plików

4. Jeśli Python nie widzi plików, sprawdź, w jakim folderze pracuje:

```
import os
print(os.getcwd())
```

5. Możecie współpracować i pomagać sobie nawzajem, to nie sprawdzian. Chciałbym, żebyście mimo to próbowali sami rozwiązywać zadania. Nie zweryfikuję tego, ale potem nie ja będę pisał maturę ;) Poza tym będę sprawdzał, co wypracowaliście. Nudno czytać n-ty raz to samo.
6. Jeśli z jakimś ćwiczeniem jest problem: występują błędy, polecenie jest niejasne itp. wyślijcie to, co uda się wypracować, tak jak zrozumieliście lub pomińcie dane zadanie. Wtedy jedna osoba niech wspomni o tym przy wysyłaniu rozwiązań. Wrócimy do zadania.
7. Przy każdym ćwiczeniu jest **oczekiwany wynik**. Jeśli Twój program wypisuje dokładnie to samo, jest dobrze. Zwracaj uwagę nawet na spacje i dwukropki, bo na maturze też liczy się dokładny format.
8. Dane „z klawiatury” podaję po strzałce: *Wejście* → ...

Co oddajesz

Wszystkie pliki `.py` z ćwiczeń wysyłasz **pod koniec zajęć** przy pomocy przycisku na stronie lub ręcznie na adres k.gebusia@uczelniakorczaka.pl. Nie zostawiaj pustych plików: **jeśli nie umiesz zrobić całego ćwiczenia, oddaj to, co masz**. Częściowe rozwiązanie jest cenne, bo pokazuje nad czym pracowałeś. Zadanie dodatkowe możesz wysłać później, do następnych zajęć.

Jeśli nie zdążysz zrobić wszystkiego na zajęciach (i **osoby nieobecne**), możesz wysłać pozostałe lub poprawione zadania po zajęciach.

Temat 1. Wejście, wyjście, formatowanie (ok. 20 min)

Teoria

input() zawsze zwraca napis (`str`), nawet jeśli wpiszesz cyfry. Żeby liczyć, trzeba zamienić go na liczbę: `int(...)` (całkowita) lub `float(...)` (rzeczywista).

```
a = input()          # wpisujesz 12 -> a to napis "12"
print(a + a)         # 1212 (sklejenie napisów!)
n = int(input())      # n to liczba 12
print(n + n)         # 24
```

print ma dwa przydatne parametry: `sep` (co wstawić między wartościami, domyślnie spacja) i `end` (co wypisać na końcu, domyślnie nowa linia).

```
print(1, 2, 3)           # 1 2 3
print(1, 2, 3, sep="-")  # 1-2-3
print("a", end="")
print("b")               # ab   (w jednej linii)
```

f-stringi to najwygodniejszy sposób wypisywania: napis poprzedzony literą `f`, a w nawiasach `{ }` wstawiasz wartości. Po dwukropku możesz podać **format**:

Format	Znaczenie	Przykład (<code>x = 3.14159</code> , <code>n = 202</code>)
<code>{x:.2f}</code>	2 miejsca po przecinku	<code>3.14</code>
<code>{n:6d}</code>	liczba całkowita na 6 znakach, wyrównana do prawej	<code>" 202"</code>
<code>{n:<6d}</code>	wyrównanie do lewej	<code>"202 "</code>
<code>{n:08b}</code>	system dwójkowy na 8 znakach, wypełnienie zerami	<code>11001010</code>
<code>{n:o}</code>	system ósemkowy	<code>312</code>
<code>{n:x}</code> / <code>{n:X}</code>	system szesnastkowy (małe / duże litery)	<code>ca</code> / <code>CA</code>

```
n = 202
print(f"Liczba {n} w dwójkowym to {n:08b}")
print(f"Pι to około {3.14159:.2f}")
```

Konwersje między systemami. Funkcje `bin()`, `oct()`, `hex()` zamieniają liczbę na napis **z przedrostkiem** (`0b`, `0o`, `0x`). Funkcja `int(napis, podstawa)` robi odwrotnie:

```
print(bin(202))          # 0b11001010
print(bin(202)[2:])      # 11001010   (obcinamy przedrostek)
print(hex(255))          # 0xff
print(int("1011", 2))    # 11
print(int("ff", 16))     # 255
```

Dzielenie i reszta. `/` daje zawsze liczbę rzeczywistą, `//` to dzielenie całkowite, `%` to reszta, `**` to potęga. `divmod(a, b)` zwraca oba wyniki naraz:

```
print(17 / 5)          # 3.4
print(17 // 5)         # 3
print(17 % 5)          # 2
print(divmod(17, 5))   # (3, 2)
print(2 ** 10)         # 1024
```

Format zagnieżdżony. Szerokość też może być zmienną. Zapamiętaj ten wzór, przyda się w ćwiczeniu 3:

```
szer = 8
print(f"{'abc':>{szer}}") # "      abc" (abc wyrównane do prawej na 8 znakach)
```

Ćwiczenia

Ćwiczenie 1 ★ (plik `t1_cw1.py`) Wczytaj dwie liczby całkowite `a` i `b` (każda w osobnej linii). Wypisz ich sumę, różnicę, iloczyn, iloraz (z dokładnością do 2 miejsc po przecinku), część całkowitą z dzielenia i resztę.

Wejście → `17`, `5`. Oczekiwany wynik:

```
Suma: 22
Różnica: 12
Iloczyn: 85
Iloraz: 3.40
Część całkowita: 3
Reszta: 2
```

Ćwiczenie 2 ★★ (plik `t1_cw2.py`) Wczytaj liczbę całkowitą z zakresu 0–255. Wypisz ją w systemie dwójkowym (zawsze 8 cyfr), ósemkowym i szesnastkowym (duże litery), używając f-stringa.

Wejście → `202`. Oczekiwany wynik:

```
Dwójkowo: 11001010
Ósemkowo: 312
Szesnastkowo: CA
```

Ćwiczenie 3 ★★★ (plik `t1_cw3.py`) Wczytaj dwie liczby zapisane w systemie **dwójkowym** (jako napisy, każda w osobnej linii). Dodaj je „pod kreską” i wypisz działanie wyrównane do prawej. Wskazówki: zamień napisy na liczby przez `int(x, 2)`, dodaj, a wynik zamień z powrotem na napis przez `bin(...)[2:]`. Szerokość kolumny to długość wyniku plus 1 (na znak `+`). Do wyrównania użyj formatu zagnieżdżonego.

Wejście → 1011 , 110 . Oczekiwany wynik (ostatnia linia ma 6 znaków, pierwsza też, licząc spacje z przodu):

```

1011
+ 110
-----
10001

```

Temat 2. Napisy (ok. 25 min)

Teoria

Napis (`str`) to ciąg znaków. Każdy znak ma **indeks**, liczony od 0. Indeksy ujemne liczą od końca (`-1` to ostatni znak).

```

s = "Python"
#   indeksy:  0 1 2 3 4 5
#   ujemne:   -6-5-4-3-2-1
print(s[0])    # P
print(s[-1])   # n
print(len(s))  # 6

```

Wycinki (slicing): `s[od:do:krok]` , gdzie „od” jest włącznie, a „do” **wyłącznie**. Każdą z trzech części można pominąć.

```

s = "Python"
print(s[1:4])    # yth
print(s[:2])     # Py      (od początku)
print(s[3:])     # hon     (do końca)
print(s[::2])    # Pto     (co drugi znak)
print(s[::-1])   # nohtyP  (odwrócony napis!)

```

Napisy są niezmiennie. Nie można zrobić `s[0] = "J"` (błąd `TypeError`). Zamiast tego stworzysz nowy napis, np. `"J" + s[1:]` .

Najważniejsze metody (żadna nie zmienia oryginału, wszystkie zwracają nowy wynik):

Metoda	Co robi	Przykład → wynik
<code>s.upper()</code> / <code>s.lower()</code>	duże / małe litery	<code>"Ala".lower()</code> → <code>"ala"</code>
<code>s.strip()</code>	usuwa spacje i znak nowej linii z początku i końca	<code>" a \n".strip()</code> → <code>"a"</code>

Metoda	Co robi	Przykład → wynik
<code>s.split()</code>	dzieli na listę po spacjach	<code>"a b c".split()</code> → <code>['a', 'b', 'c']</code>
<code>s.split(";")</code>	dzieli po podanym znaku	<code>"x;y;z".split(";")</code> → <code>['x', 'y', 'z']</code>
<code>"-".join(lista)</code>	skleja listę napisów, wstawiając -	<code>"-".join(["a", "b"])</code> → <code>"a-b"</code>
<code>s.count("a")</code>	ile razy występuje	<code>"banan".count("a")</code> → 2
<code>s.find("na")</code>	indeks pierwszego wystąpienia lub -1	<code>"banan".find("na")</code> → 2
<code>s.replace("a", "o")</code>	zamienia wszystkie wystąpienia	<code>"banan".replace("a", "o")</code> → <code>"bonon"</code>
<code>s.startswith("ba")</code>	czy zaczyna się od	<code>True</code>
<code>s.isdigit()</code> / <code>s.isalpha()</code>	same cyfry / same litery	<code>"123".isdigit()</code> → <code>True</code>

Operator `in` sprawdza, czy fragment występuje: `"yth" in "Python"` daje `True`. Po napisie można iterować pętlą `for`:

```
for znak in "abc":
    print(znak)          # a, b, c w osobnych liniach
```

Kody znaków. `ord(znak)` zwraca numer znaku w tabeli (np. `ord("a")` to 97), a `chr(numer)` robi odwrotnie. Litery alfabetu mają kolejne numery, więc można na nich „liczyć”:

```
print(ord("a"))          # 97
print(chr(98))           # b
print(ord("d") - ord("a")) # 3 (numer litery w alfabecie, licząc od 0)
```

Reszta z dzielenia przy „zawijaniu” alfabetu. Aby po `z` wrócić do `a`, używamy `% 26`:

```
# przesunięcie litery o 3 miejsca z zawinięciem
litera = "y"
nowa = chr((ord(litera) - ord("a") + 3) % 26 + ord("a"))
print(nowa)      # b
```

Ćwiczenia

Ćwiczenie 1 ★ (plik `t2_cw1.py`) Wczytaj słowo (usuń zbędne spacje przez `strip()` i zamień na małe litery przez `lower()`). Wypisz jego długość, pierwszą i ostatnią literę, słowo odwrócone oraz informację, czy jest palindromem (czyta się tak samo w obie strony).

Wejście → `kajak`:

```
Długość: 5
Pierwsza litera: k
Ostatnia litera: k
Odwrócone: kajak
Palindrom: TAK
```

Wejście → `Python`:

```
Długość: 6
Pierwsza litera: p
Ostatnia litera: n
Odwrócone: nohtyp
Palindrom: NIE
```

Ćwiczenie 2 ★★ (plik `t2_cw2.py`) Wczytaj adres IPv4 w postaci `192.168.1.10`. Rozbij go po kropkach (`split(".")`), każdą część zamień na liczbę i zapisz jako **8 bitów** (format `08b`). Wypisz wynik jako cztery ośmioznakowe bloki połączone kropkami (`join`).

Wejście → `192.168.1.10`:

```
11000000.10101000.00000001.00001010
```

Ćwiczenie 3 ★★★ (plik `t2_cw3.py`) Szyfr Cezara: każdą literę słowa przesuwamy w alfabecie o `k` miejsc. Wczytaj słowo złożone z małych liter `a - z` (w pierwszej linii) i przesunięcie `k` (w drugiej). Wypisz słowo zaszyfrowane, a potem odszyfrowane (przesunięcie o `-k`), żeby sprawdzić, czy wraca oryginał. Użyj wzoru z teorii. Budowanie wyniku: zacznij od pustego napisu `wynik = ""` i w pętli dołączaj znaki: `wynik += ...`.

Wejście → `matura`, `3`:

```
Zaszyfrowane: pdwxud
Odszyfrowane: matura
```

Wejście → `xyz`, `3` (sprawdza zawijanie alfabetu):

```
Zaszyfrowane: abc
Odszyfrowane: xyz
```

Temat 3. Pliki (ok. 30 min)

Teoria

Na maturze dane prawie zawsze są w plikach tekstowych. Standardowy sposób odczytu:

```
with open("liczby.txt", encoding="utf-8") as f:
    for linia in f:
        print(linia)
```

Co tu się dzieje:

- `open(nazwa, tryb, encoding=...)` otwiera plik. Tryby: `"r"` odczyt (domyślny), `"w"` zapis (**kasuje** starą zawartość!), `"a"` dopisywanie na końcu.
- `with ... as f:` sam zamknie plik po wyjściu z bloku.
- `encoding="utf-8"` jest ważne, bo w plikach są polskie litery (ą, ę, ł...). Bez tego na Windowsie mogą pojawić się „krzaczki” lub błąd.
- `for linia in f` daje kolejne linie jako napisy.

Uwaga na znak nowej linii. Każda wczytana linia kończy się znakiem `\n`. Dlatego powyższy kod wypisze puste linie między liczbami. Usuń go przez `strip()`:

```
with open("liczby.txt", encoding="utf-8") as f:
    for linia in f:
        print(linia.strip())
```

Wczytanie liczb do listy (plik z jedną liczbą w linii). Wzorec do zapamiętania:

```
liczby = []
with open("liczby.txt", encoding="utf-8") as f:
    for linia in f:
        liczby.append(int(linia.strip()))
print(liczby)
print(len(liczby), sum(liczby))
```

(`int(" 12\n")` samo ignoruje białe znaki, więc `strip()` nie jest tu konieczny, ale to dobry nawyk).

Linia z kilkoma wartościami. Plik `uczniowie.txt` ma linie postaci `Anna;Kowalska;3A;87`. Rozbijamy je po średniku i **rozpakowujemy** do kilku zmiennych naraz:

```
with open("uczniowie.txt", encoding="utf-8") as f:
    for linia in f:
        imie, nazwisko, klasa, punkty = linia.strip().split(";")
        print(nazwisko, int(punkty) + 1)
```

Liczba zmiennych po lewej musi się zgadzać z liczbą części, inaczej `ValueError`.

Plik `dane liniowe.txt` ma w każdej linii 5 liczb oddzielonych spacją. Zamiana linii na listę liczb:

```
linia = "4 8 15 16 23"
liczby = list(map(int, linia.split())) # [4, 8, 15, 16, 23]
```

`map(int, ...)` stosuje `int` do każdego elementu, a `list(...)` zamienia wynik na listę. Możesz też to zrobić zwykłą pętlą, jeśli wolisz.

Zapis do pliku. Pamiętaj, że `write` **nie dodaje** nowej linii, dopisz `\n` sam. Możesz też użyć `print(..., file=out)`, które dodaje ją automatycznie:

```
with open("wynik.txt", "w", encoding="utf-8") as out:
    out.write("pierwsza linia\n")
    print("druga linia", file=out)
```

Jeśli potrzebujesz czytać z jednego pliku i pisać do drugiego jednocześnie, otwórz oba w jednym `with`, rozdzielając przecinkiem:

```
with open("a.txt", encoding="utf-8") as f, open("b.txt", "w", encoding="utf-8") as out:
    ...
```

Ćwiczenia

Ćwiczenie 1 ★ (plik `t3_cw1.py`) Wczytaj liczby z `liczby.txt` do listy. Wypisz: ile ich jest, ich sumę, najmniejszą, największą oraz średnią (2 miejsca po przecinku).

Oczekiwany wynik:

```
Ile liczb: 15
Suma: 569
Najmniejsza: 3
Największa: 90
Średnia: 37.93
```

Ćwiczenie 2 ★★ (plik `t3_cw2.py`) Plik `uczniowie.txt` zawiera: imię, nazwisko, klasę i punkty (oddzielone średnikami). Wypisz na ekran wszystkie osoby z co najmniej **70 punktami** w postaci `Nazwisko Imię (klasa): punkty`. Te same osoby (tylko `Nazwisko Imię`) zapisz do pliku `wyniki.txt`. Na końcu wypisz liczbę takich osób i średnią punktów **wszystkich** uczniów.

Oczekiwany wynik na ekranie:

```
Kowalska Anna (3A): 87
Żak Łukasz (3A): 71
Ćwik Zofia (3C): 92
Dąbrowska Maria (3A): 78
Zielińska Ewa (3B): 83
Wójcik Katarzyna (3C): 70
Liczba osób: 6
Średnia wszystkich: 71.60
```

Plik `wyniki.txt` ma mieć 6 linii: `Kowalska Anna`, `Żak Łukasz`, itd.

Ćwiczenie 3 ★★★ (plik `t3_cw3.py`) Plik `dane liniowe.txt` ma 6 linii po 5 liczb. Dla każdej linii policz sumę jej liczb, wypisz ją jako `Linia nr: suma` (numeracja od 1) i zapisz do pliku `sumy.txt` (jedna suma w linii). Na końcu wypisz numer linii o największej sumie. Podpowiedź: licznik linii zwiększaj w pętli, a „rekordzistę” zapamiętuj w zmiennych (najlepsza suma i jej numer), tak jak szuka się maksimum.

Oczekiwany wynik:

```
Linia 1: 66
Linia 2: 72
Linia 3: 114
Linia 4: 30
Linia 5: 183
Linia 6: 26
Największa suma w linii nr 5
```

Temat 4. Listy i krotki (ok. 25 min)

Teoria

Lista to uporządkowany, **zmienny** zbiór elementów. Indeksowanie i wycinki działają jak w napisach (patrz temat 2).

```

a = [5, 3, 8]
a.append(1)          # dodaj na koniec          -> [5, 3, 8, 1]
a.extend([7, 7])     # dodaj kilka na koniec    -> [5, 3, 8, 1, 7, 7]
a.insert(0, 9)       # wstaw na pozycję 0      -> [9, 5, 3, 8, 1, 7, 7]
x = a.pop()          # zdejmij ostatni (x = 7)
a.pop(0)             # zdejmij element o indeksie 0
a.remove(8)          # usuń pierwszą 8 (po wartości)
print(a.index(3))    # indeks pierwszego 3
print(3 in a)        # czy 3 jest na liście

```

Funkcje na listach: `len(a)`, `min(a)`, `max(a)`, `sum(a)`.

Sortowanie. Dwie wersje, nie pomył ich:

```

a = [5, 3, 8]
b = sorted(a)        # zwraca NOWĄ posortowaną listę, a bez zmian
a.sort()             # sortuje a „w miejscu” i zwraca None
print(sorted(a, reverse=True)) # malejąco

```

Krotka (tuple) to lista, której nie można zmieniać: `t = (1, 2, 3)`. Próba `t[0] = 9` kończy się błędem `TypeError`.

Rozpakowywanie pozwala przypisać elementy do kilku zmiennych naraz. Dzięki temu zamiana wartości jest bardzo prosta:

```

x, y = 3, 7
x, y = y, x          # zamiana, teraz x = 7, y = 3
l = [10, 20, 30]
A, B, C = l           # A=10, B=20, C=30
l[0], l[2] = l[2], l[0] # zamiana elementów listy o indeksach 0 i 2

```

PUŁAPKA 1: kopiowanie list. Zapis `b = a` **nie kopiuje** listy, tylko tworzy drugą „etykietę” na tę samą listę:

```

a = [1, 2, 3]
b = a                # b i a to TA SAMA lista
b.append(4)
print(a)             # [1, 2, 3, 4] <- a też się zmieniło!

c = a[:]             # prawdziwa kopia (wycinek od początku do końca)
# albo c = a.copy()
c.append(5)
print(a)             # [1, 2, 3, 4] <- a bez zmian

```

PUŁAPKA 2: tablica dwuwymiarowa. Lista list to „tablica 2D”. Zły sposób jej tworzenia:

```
m = [[0] * 3] * 2      # ŹLE: oba wiersze to ta sama lista
m[0][0] = 5
print(m)                # [[5, 0, 0], [5, 0, 0]] <- zmieniły się oba wiersze!
```

Poprawny sposób (zapamiętaj ten zapis jako wzorzec, dokładnie wyjaśnimy go na kolejnych zajęciach):

```
m = [[0] * 3 for _ in range(2)] # DOBRZE: każdy wiersz osobny
m[0][0] = 5
print(m)                        # [[5, 0, 0], [0, 0, 0]]
```

Ćwiczenia

Ćwiczenie 1 ★: przewidź wynik (plik `t4_cw1.py`) Zanim uruchomisz kod, **zapisz w komentarzu (# ...) obok każdej linii, co Twoim zdaniem zostanie wypisane**. Potem uruchom i porównaj. Każdą pomyłkę potraktuj jako informację, co jeszcze trzeba zrozumieć. Przepisz kod do pliku:

```
a = [1, 2, 3]; b = a; b.append(4); print(a)
a = [1, 2, 3]; b = a[:]; b.append(4); print(a)
m = [[0] * 3] * 2; m[0][0] = 5; print(m)
m = [[0] * 3 for _ in range(2)]; m[0][0] = 5; print(m)
x, y = 3, 7; x, y = y, x; print(x, y)
a = [10, 20, 30, 40, 50]; print(a[1:4], a[-2:], a[::-2])
```

Poprawne wyniki (zastój je, dopóki nie zapiszesz swoich przewidywań!):

```
[1, 2, 3, 4]
[1, 2, 3]
[[5, 0, 0], [5, 0, 0]]
[[5, 0, 0], [0, 0, 0]]
7 3
[20, 30, 40] [40, 50] [10, 30, 50]
```

Ćwiczenie 2 ★★ (plik `t4_cw2.py`) Dana lista `a = [7, 3, 12, 1, 9, 4, 8, 2]`. Wypisz: najmniejszy i największy element, **drugi największy** (podpowiedź: posortowana kopia i indeks `-2`), liczbę elementów parzystych, posortowaną kopię oraz oryginał (który **ma pozostać niezmienny**, więc użyj `sorted`, a nie `sort`).

Oczekiwany wynik:

```

Najmniejszy: 1
Największy: 12
Drugi największy: 9
Parzystych: 4
Posortowana kopia: [1, 2, 3, 4, 7, 8, 9, 12]
Oryginał: [7, 3, 12, 1, 9, 4, 8, 2]

```

Ćwiczenie 3 ★★ (plik `t4_cw3.py`) Przesuń cyklicznie listę `a = [1, 2, 3, 4, 5, 6, 7]` o `k = 3` miejsc w lewo (pierwsze `k` elementów ma trafić na koniec), a następnie o 3 miejsca w prawo (ostatnie `k` elementów trafia na początek). Użyj tylko wycinków i łączenia list znakiem `+`. Nie używaj pętli.

Oczekiwany wynik:

```

W lewo: [4, 5, 6, 7, 1, 2, 3]
W prawo: [5, 6, 7, 1, 2, 3, 4]

```

Ćwiczenie 4 ★★★ (plik `s1_t4_cw4.py`) Wczytaj liczby z `liczby.txt` do listy. Wypisz listę, potem znajdź indeks najmniejszego i największego elementu (metoda `index`), zamień je miejscami przez rozpakowywanie (`l[i], l[j] = l[j], l[i]`) i wypisz listę po zmianie.

Oczekiwany wynik:

```

Przed: [12, 7, 45, 3, 88, 21, 56, 9, 34, 67, 5, 90, 18, 73, 41]
Indeks najmniejszej: 3 indeks największej: 11
Po:      [12, 7, 45, 90, 88, 21, 56, 9, 34, 67, 5, 3, 18, 73, 41]

```

Temat 5. Pętle i wygodne konstrukcje (ok. 20 min)

Teoria

`range(start, stop, krok)`: `stop` jest wyłączone, `krok` może być ujemny.

```

print(list(range(5)))      # [0, 1, 2, 3, 4]
print(list(range(2, 8)))   # [2, 3, 4, 5, 6, 7]
print(list(range(10, 0, -3))) # [10, 7, 4, 1]

```

Gwiazdka przy `print` rozpakowuje listę na osobne argumenty:

```

print(*range(5))    # 0 1 2 3 4
print(*[1, 2, 3], sep=", ") # 1, 2, 3

```

enumerate daje jednocześnie numer i element, więc nie trzeba pisać `range(len(...))` ani ręcznego licznika:

```
owoce = ["jabłko", "gruszka", "śliwka"]
for i, o in enumerate(owoce):          # numeracja od 0
    print(i, o)
for i, o in enumerate(owoce, start=1): # numeracja od 1
    print(i, o)
```

zip przechodzi równoległe przez kilka list:

```
imiona = ["Anna", "Jan"]
wieki = [17, 18]
for imie, wiek in zip(imiona, wieki):
    print(imie, wiek)
```

Można je łączyć: `for nr, (imie, wiek) in enumerate(zip(imiona, wieki), start=1):`. Nawias przy `(imie, wiek)` jest konieczny.

break i continue. `break` przerywa całą pętlę, `continue` pomija resztę bieżącego obrotu i przechodzi do następnego:

```
for i in range(10):
    if i % 2 == 0:
        continue      # pominięcie parzyste
    if i > 7:
        break          # zakończ pętlę
    print(i)           # 1 3 5 7
```

for ... else. Blok `else` po pętli wykona się **tylko wtedy, gdy pętla nie została przerwana przez break**. To idealne do sprawdzania, czy „nic nie znaleziono”, np. czy liczba jest pierwsza:

```
n = 13
for d in range(2, n):          # czy można zoptymalizować zakres?
    if n % d == 0:
        print(n, "ma dzielnik", d)
        break
else:
    print(n, "jest pierwsza")  # wykona się, bo nie było break
```

Pętla while powtarza się, dopóki warunek jest prawdziwy. Używamy jej, gdy nie wiemy z góry, ile będzie obrotów. Pamiętaj, by w środku zmieniać coś, co kiedyś zakończy pętlę (inaczej będzie nieskończona; przerwij ją klawiszami Ctrl+C):

```
n = 100
kroki = 0
while n > 1:
    n //= 2      # to samo co n = n // 2
    kroki += 1
print(kroki)    # 6 (ile razy można podzielić 100 przez 2, zanim zostanie 1)
```

Ćwiczenia

Ćwiczenie 1 ★ (plik `t5_cw1.py`) (a) Wypisz w jednej linii liczby od 100 do 0 co 10 (użyj `range` z ujemnym krokiem i `print(...)`). (b) Policz sumę wszystkich liczb od 0 do 999 podzielnych przez 3 **lub** przez 5.

Oczekiwany wynik:

```
100 90 80 70 60 50 40 30 20 10 0
Suma: 233168
```

Ćwiczenie 2 ★★ (plik `t5_cw2.py`) (a) Dane są listy `imiona = ["Anna", "Jan", "Zofia"]` i `punkty = [87, 64, 92]`. Wypisz ponumerowaną listę (numeracja od 1), używając `enumerate` i `zip`. (b) Wczytaj `liczby.txt` do listy i znajdź **pierwszą** liczbę większą od 50 oraz jej pozycję (numerowaną od 1). Użyj `break`, a na wypadek braku takiej liczby dodaj `else` z komunikatem `Brak takiej liczby`.

Oczekiwany wynik:

```
1. Anna - 87
2. Jan - 64
3. Zofia - 92
Pierwsza liczba większa od 50: 88 (pozycja 5)
```

Ćwiczenie 3 ★★★ (plik `t5_cw3.py`) (a) Zbierz do listy wszystkie liczby pierwsze od 2 do 100 (użyj konstrukcji `for ... else` z teorii, a w `else` dodawaj liczbę do listy), wypisz je w jednej linii i podaj ich liczbę. (b) **Ciąg Collatza**: zaczynając od `n = 27`, powtarzaj: jeśli `n` jest parzyste, podziel je przez 2, w przeciwnym razie zamień na `3*n + 1`. Kończysz, gdy `n` osiągnie 1. Policz, ile było kroków i jaka największa wartość pojawiła się po drodze.

Oczekiwany wynik:

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Ile: 25
Kroki: 111 Maksimum: 9232
```

Zadanie dodatkowe: analiza tekstu (w domu)

To zadanie jest dłuższe i wzorowane na maturalnym: kilka podpunktów, dane w pliku, wynik zapisany do pliku. **Nie ma tu oczekiwanych wyników**, bo sprawdzanie poprawności jest częścią zadania. Zrób wszystkie podpunkty w jednym pliku `dodatkowe.py`.

Plik `tekst.txt` zawiera dwa akapity tekstu po polsku. Napisz program, który:

1. Wczytuje cały tekst i dzieli go na słowa. Słowa mają być pozbawione znaków interpunkcyjnych (`.,;:!?,` użyj `strip(".,;:!?")`) i zapisane małymi literami.
2. Wyznacza **liczbę słów** w tekście.
3. Znajduje **najdłuższe słowo** (jeśli jest kilka o tej samej długości, wybierz to, które występuje w tekście jako pierwsze) i jego długość.
4. Znajduje wszystkie **palindromy** o długości co najmniej 3 liter. Każdy palindrom wypisz tylko raz, w kolejności alfabetycznej (podpowiedź: `if s not in lista`, potem `lista.sort()`).
5. Liczy, ile razy w całym tekście występuje litera **a** (łącznie we wszystkich słowach, metoda `count`).
6. Zapisuje wyniki do pliku `raport.txt`, po jednej odpowiedzi w linii, w postaci:

```
Liczba słów: ...
Najdłuższe słowo: ... (... liter)
Palindromy: ..., ..., ...
Liczba liter a: ...
```

Wskazówki:

- Zacznij od wypisania na ekran listy słów i sprawdź, czy wygląda sensownie, zanim policzysz cokolwiek.
- Rób po jednym podpunkcie i uruchamiaj program po każdym, zamiast pisać wszystko naraz.
- Jeśli coś nie działa, wypisz wartości pośrednie (`print`), żeby zobaczyć, co naprawdę jest w zmiennych.
- Na maturze za zadanie z kilkoma podpunktami dostaje się punkty za każdy z nich osobno, więc działający podpunkt 2 jest wart więcej niż niedokończone wszystko.

Ściągawka z zajęć

Potrzebuję...	Używam...
liczbę z klawiatury	<code>n = int(input())</code>

Potrzebuję...	Używam...
wynik z 2 miejscami po przecinku	<code>f"{x:.2f}"</code>
liczbę w systemie dwójkowym (8 bitów)	<code>f"{n:08b}"</code>
liczbę z napisu w systemie 2 / 16	<code>int(s, 2) / int(s, 16)</code>
odwrócony napis	<code>s[::-1]</code>
napis rozbić / skleić	<code>s.split(";") / ";".join(lista)</code>
numer litery w alfabecie	<code>ord(z) - ord("a")</code>
wczytać plik linia po linii	<code>with open("p.txt", encoding="utf-8") as f: + for linia in f:</code>
linię liczb zamienić na listę	<code>list(map(int, linia.split()))</code>
prawdziwa kopia listy	<code>b = a[:] lub a.copy()</code>
posortowana kopia (bez zmiany oryginału)	<code>sorted(a)</code>
zamiana dwóch wartości	<code>x, y = y, x</code>
numer i element jednocześnie	<code>for i, x in enumerate(a):</code>
dwie listy równoległe	<code>for x, y in zip(a, b):</code>
„nic nie znaleziono” po pętli	<code>for ... else</code>